Университет ИТМО
Мегафакультет трансляционных информационных технологий
Факультет информационных технологий и программирования

Группа: M3206

К работе допущен:

Студент: Абубакиров Аскар

Работа выполнена:

Преподаватель: Гумбатов Владислав Юрьевич

Отчет принят:

Рабочий протокол и отчет по лабораторной работе № 3

Демонизация, systemd unit, планирование задач, сигналы и просмотр логов в Linux

---

```bash
#!/bin/bash

EVENT_FILE="data/events.log"
LOG_FILE="logs/watcher.log"
PID_FILE="run/watcher.pid"
FIFO_FILE="run/watcher.fifo"

EVENT_COUNT=0
RUNNING=1

MODE="normal"

log() {
        local level="$1"
        local message="$2"

        echo "[$(date '++%F %T')] [$level] $message" >> "$LOG_FILE"
}

handle_event() {
        local event="$1"
        EVENT_COUNT=$((EVEN_COUNT + 1))
        log "EVENT" "received event #$EVENT_COUNT: $event"
}

on_term() {
        log "SIGNAL" "received SIGTERM, stopping watcher"
        RUNNING=0
}

on_int() {
        log "SIGNAL" "received SIGINT, stopping watcher"
        RUNNING=0
}

on_hup() {
        log "SIGNAL" "received SUGHUP, reload request"
}

on_usr1() {
        log "SIGNAL" "received SIGUSR1, status requested, processed=$EVENT_COUNT"
}
```

```bash
on_usr2() {
        log "SIGNAL" "received SIGUSR2, custom action requested"
}

is_running() {
        if [ -f "$PID_FILE" ]; then
                local pid
                pid=$(cat "$PID_FILE")

                if kill -0 "$pid" 2>/dev/null; then
                        return 0
                fi
        fi
        return 1
}
```

```
handle_fifo() {
        local cmd="$1"

        cmd=$(echo "$cmd" | xargs)

        if [ -z "$cmd" ]; then
                log "FIFO" "empty input received"
                return
        fi

        case "$cmd" in
                STOP)
                        log "FIFO" "received STOP command"
                        RUNNING=0
                        ;;
                STATUS)
                        log "FIFO" "status requested: pid=$$, processed=$EVENT_COUNT"
                        ;;
                MODE_CHANGE)
                        if [ "$MODE" = "normal" ]; then
                                MODE="verbose"
                        else
                                MODE="normal"
                        fi

                        log "FIFO" "mode changed to $MODE"
                        ;;
                RELOAD)
                        log "FIFO" "reload requested"
                        ;;
                *)
                        log "FIFO" "unknown command: $cmd"
```

```
                        ;;
                *)
                        log "FIFO" "unknown command: $cmd"
                        ;;
        esac
}

run_watcher() {
        trap on_term TERM
        trap on_int INT
        trap on_hup HUP
        trap on_usr1 USR1
        trap on_usr2 USR2

        mkdir -p logs data run
        touch "$EVENT_FILE" "$LOG_FILE"
        mkfifo -m 666 "$FIFO_FILE" 2>/dev/null

        echo $$ > "$PID_FILE"
        log "INFO" "watcher started, pid=$$"

        exec 3< <(tail -n 0 -F "$EVENT_FILE")
        exec 4<>"$FIFO_FILE"

        while [ "$RUNNING" -eq 1 ]; do
                if read -t 1 -r line <&3; then
```

```
run_watcher() {
        trap on_term TERM
        trap on_int INT
        trap on_hup HUP
        trap on_usr1 USR1
        trap on_usr2 USR2

        mkdir -p logs data run
        touch "$EVENT_FILE" "$LOG_FILE"
        mkfifo -m 666 "$FIFO_FILE" 2>/dev/null

        echo $$ > "$PID_FILE"
        log "INFO" "watcher started, pid=$$"

        exec 3< <(tail -n 0 -F "$EVENT_FILE")
        exec 4<>"$FIFO_FILE"

        while [ "$RUNNING" -eq 1 ]; do
                if read -t 1 -r line <&3; then
                        handle_event "$line"
                fi

                if read -t 0.1 -r cmd <&4; then
                        handle_fifo "$cmd"
                fi
        done

        log "INFO" "watcher stopped, pid=$$"
        rm -f "$PID_FILE"
}
```

```
start_watcher() {
        if is_running; then
                echo "Watcher already running with PID $(cat "$PID_FILE")"
                exit 1
        fi

        nohup "$0" run>/dev/null 2>&1 &
        sleep 1

        if is_running; then
                echo "Watcher started with PID $(cat "$PID_FILE")"
        else
                echo "Failed to start watcher"
                exit 1
        fi
}
```

```
stop_watcher() {
        if ! is_running; then
                echo "Watcher is not running"
                rm -f "$PID_FILE"
                exit 1
        fi

        local pid
        pid=$(cat "$PID_FILE")
        kill -TERM "$pid"

        for i in {1..10}; do
                if kill -0 "$pid" 2>/dev/null; then
                        sleep 1
                else
                        break
                fi
        done

        rm -f "$PID_FILE"
        echo "Watcher stopped"
}

status_watcher() {
        if is_running; then
                echo "Watcher is running with PID $(cat "$PID_FILE")"
        else
                echo "Watcher is not running"
        fi
}
```

```
generate_report() {
        mkdir -p reports

        local report_file="reports/report_$(date '+%F').txt"

        {
                echo "Watcher daily report"
                echo "Generated: $(date '+%F %T')"
                echo "----------------------------------------"
                echo "Total EVENT records: $(grep -c '\[EVENT\]' "$LOG_FILE" 2>/dev/null)"
                echo "Total FIFO records: $(grep -c '\[FIFO\]' "$LOG_FILE" 2>/dev/null)"
                echo "Total SIGNAL records: $(grep -c '\[SIGNAL\]' "$LOG_FILE" 2>/dev/null)"
                echo "Total INFO records: $(grep -c '\[INFO\]' "$LOG_FILE" 2>/dev/null)"
                echo "----------------------------------------"
                echo "Last 10 log lines:"
                tail -n 10 "$LOG_FILE" 2>/dev/null
        } > "$report_file"

        echo "Report created: $report_file"
}

cleanup_reports() {
        find reports -type f -name "reports_*.txt" -mtime +7 -delete
        echo "Old reports cleanup completed"
}
```

```bash
case "$1" in
        run)
                run_watcher
                ;;
        start)
                start_watcher
                ;;
        stop)
                stop_watcher
                ;;
        status)
                status_watcher
                ;;
        report)
                generate_report
                ;;
        cleanup)
                cleanup_reports
                ;;
        *)
                exit 1
                ;;
esac
```

```
[root@localhost lab3]# systemctl status watcher
● watcher.service - Process Watcher Service
   Loaded: loaded (/etc/systemd/system/watcher.service; enabled; vendor preset: disabled)
   Active: active (running) since Thu 2026-03-26 14:45:19 MSK; 1min 21s ago
 Main PID: 1794 (bash)
    Tasks: 2 (limit: 23966)
   Memory: 1.1M
   CGroup: /system.slice/watcher.service
           ├─1794 /bin/bash /home/user/lab3/watcher.sh run
           └─1801 tail -n 0 -F data/events.log

Mar 26 14:45:19 localhost.localdomain systemd[1]: Started Process Watcher Service.
[root@localhost lab3]# kill -USR1 1794
[root@localhost lab3]# kill -HUP 1794
[root@localhost lab3]# kill -USR2 1794
[root@localhost lab3]# echo "STATUS" > run/watcher.fifo
[root@localhost lab3]# echo "event test 1" >> data/events.log
[root@localhost lab3]# cat logs/watcher.log
[+2026-03-26 14:45:19] [SIGNAL] received SIGTERM, stopping watcher
[+2026-03-26 14:45:19] [INFO] watcher stopped, pid=887
[+2026-03-26 14:45:19] [INFO] watcher started, pid=1794
[+2026-03-26 14:46:20] [SIGNAL] received SIGUSR1, status requested, processed=0
[+2026-03-26 14:46:46] [SIGNAL] received SIGUSR1, status requested, processed=0
[+2026-03-26 14:46:51] [SIGNAL] received SUGHUP, reload request
[+2026-03-26 14:46:53] [SIGNAL] received SIGUSR2, custom action requested
[+2026-03-26 14:47:14] [FIFO] status requested: pid=1794, processed=0
[+2026-03-26 14:47:30] [EVENT] received event #1: event test 1
```

```
[root@localhost lab3]# systemctl stop watcher
[root@localhost lab3]# systemctl status watcher
● watcher.service - Process Watcher Service
   Loaded: loaded (/etc/systemd/system/watcher.service; enabled; vendor preset: disabled)
   Active: inactive (dead) since Thu 2026-03-26 14:48:15 MSK; 4s ago
  Process: 1794 ExecStart=/bin/bash /home/user/lab3/watcher.sh run (code=exited, status=0/SUCCESS)
 Main PID: 1794 (code=exited, status=0/SUCCESS)

Mar 26 14:45:19 localhost.localdomain systemd[1]: Started Process Watcher Service.
Mar 26 14:48:15 localhost.localdomain systemd[1]: Stopping Process Watcher Service...
Mar 26 14:48:15 localhost.localdomain systemd[1]: Stopped Process Watcher Service.
[root@localhost lab3]#
```